ChatGPT

GeoSee Phase 1 Diagnosis Research Summary

Current status of this summary

This summary is a **research-backed Phase 1 framing memo**, not a reproduced defect inventory from the live repo. The attached Phase 1 brief requires a diagnosis-only pass with **live evidence**, a clean list of what is **not** broken, verification of already-landed fixes such as DEC-164 and DEC-165, and then a **harden-vs-rewrite** recommendation grounded in that inventory. It also requires page-agnostic reporting and explicitly treats the regression-test net as a core deliverable. Because the repository contents and running app were not available inside the current runtime, no defect below is claimed as “reproduced”; the useful work that could be done here was to extract the exact decision standard from the prompt and map it to external best practices for animation runtime design and deterministic regression testing. filecite turn0file0 L11-L22 filecite turn0file0 L112-L149 filecite turn0file0 L153-L178

The strongest current takeaway is that the attached brief is already pointing toward the right decision process: **do not greenfield the transition engine first**. The prompt explicitly says commissioning architecture before understanding the residual defect rate is backwards, and that Phase 1 is meant to converge on shared ground truth before any design pass begins. On that basis, the best current call is **provisional incremental hardening**, not rewrite, until live evidence proves otherwise.
filecite turn0file0 L26-L31

What the attached Phase 1 brief actually requires

The brief is much stricter than a normal code audit. It asks two evidence-based questions only: **what transition behavior is still broken right now** after recent fixes, and **whether the smallest sufficient path is incremental hardening or re-architecture**. It explicitly forbids designing a new transition engine during Phase 1. That means any useful Phase 1 output must separate three things cleanly: reproduced defects, verified clean behavior, and the design recommendation that follows from that evidence.
filecite turn0file0 L11-L22

The brief also defines the observability surface. It names the active repo, the allowed read-only run path, the live runtime hooks for page navigation and scene diagnostics, and the deck-state tools that may be used only as deck-state evidence rather than broad “app health” proof. That is important because it implies Phase 1 is not a speculative architecture review; it is intended to be a disciplined runtime diagnosis with introspection, traces, screenshots, matrix captures, and numeric checks. filecite turn0file0 L35-L51

The success criteria are also unusually concrete. The prompt says the known-good reference includes, among other things, a local MT resistivity slider interpolation path, a protected MT full-density scalar-cloud path, section frames that **move** rather than dissolve when identity survives, right-handed section helper basis, composed ISO opacity, discrete material/color channels, single commit ownership, deferred-flush suppression, and overlays plus camera on the same transition clock. It also says a defect must be filed against a **behavior family**, not a page pair. That combination of requirements strongly suggests the correct

Phase 1 outcome is a per-family regression net with invariants, not a page-by-page bug list.
turn0file0L71-L92

Best-practice findings that matter most for GeoSee

One transition clock should own visible progress

Browser animation best practice is clear: `requestAnimationFrame()` is a repaint-aligned, one-shot callback whose progress should be computed from the supplied frame timestamp, otherwise animation speed will drift on high-refresh-rate displays. MDN also notes that the callback timestamp is shared across same-origin windows and iframes, which is a useful architectural signal: if GeoSee wants camera, overlays, labels, legends, section frames, and entity opacity to feel like one transition, they should all resolve from a **single transition clock** rather than mixed timers or subsystem-local elapsed-time counters. [1]

That aligns directly with the prompt's own invariant that overlays and camera should live on the transition clock rather than separate timing paths. In practice, the combination of the prompt and browser guidance points toward a simple runtime test: every visible transition family should be provable as a function of one timestamped progress scalar, and interruption/replan should update that ownership explicitly rather than allowing multiple concurrent clocks. turn0file0L86-L90 [2]

Composition rules should be explicit per channel

The Web Animations specification offers a strong analogy for transition-runtime design. It defines an **effect stack** per property and says final values are produced by combining effect values in composite order. It also formalizes composite operations such as `replace`, `add`, and `accumulate`, and distinguishes **discrete** properties from genuinely interpolable ones. Discrete properties are specifically described as not additive, with a step-like swap rather than continuous interpolation. [3]

That maps cleanly onto the GeoSee prompt. The prompt's "surviving frames interpolate, changed/added/removed dissolve" rule is, in effect, a composition policy. Its note that discrete color/material channels matter is another composition rule. The practical implication is that GeoSee's transition runtime should continue to treat different channels differently: transforms, extents, and some opacity paths can interpolate; identity, incompatible materials, helper basis changes, and other discrete channels should be routed through replace-or-dissolve semantics rather than forced through a universal tween path. turn0file0L83-L90 [4]

Interruption and replan should be modeled as explicit lifecycle states

W3C's SCXML recommendation describes an event-based state-machine model built from states, transitions, `onentry`, `onexit`, and explicit transition selection semantics; it also supports parallel states when multiple regions must advance together. For a transition runtime, that is a strong best-practice argument for putting interruption, deferred flushes, settle, commit ownership, and replan under one explicit lifecycle model instead of letting those responsibilities leak across unrelated scene and UI subsystems. [5]

This matters because several of the prompt's listed invariants are lifecycle invariants rather than geometry invariants: single commit owner, deferred-flush suppression, readiness/attach behavior, interruption/replan

correctness, and no commit-time replay pop. Those are exactly the areas where an explicit state machine tends to outperform ad hoc booleans and handoffs. That does **not** automatically mean a rewrite is needed; it means the current runtime should be evaluated against explicit ownership and transition-state semantics when Phase 1 reproductions are gathered. 【filecite】turn0file0】L86-L90】 【filecite】turn0file0】L116-L130】 [5]

Helper objects should be stable and updated, not rebuilt opportunistically

The prompt's helper-related invariants point to stability: no per-tick helper rebuild, right-handed helper basis, and numeric verification such as determinant and world bounding box checks. three.js's own `PlaneHelper` documentation reinforces that shape of solution: it updates itself through `updateMatrixWorld()` based on its current plane and size, and it exposes `dispose()` for GPU-related cleanup. In other words, helper objects are expected to be **long-lived runtime objects with controlled updates and explicit lifecycle cleanup**, not constantly recreated throwaways. 【filecite】turn0file0】L83-L90】 【filecite】turn0file0】L100-L108】 [6]

The regression-test net should be the main durable deliverable

The attached brief is explicit that the regression-test net should be treated as the durable mechanism that prevents future breakage. External testing guidance strongly supports that stance. Playwright recommends testing **user-visible behavior**, isolating tests from each other, and avoiding dependency on third-party systems outside the tester's control. Its fixtures are explicitly isolated between tests, which is the right property for reproducing transition families without state leakage. 【filecite】turn0file0】L147-L149】 [7]

For time-dependent runtime behavior, Playwright's clock support is especially relevant. It can override `Date`, timeouts, intervals, `requestAnimationFrame`, `performance`, and related browser timing APIs, and it supports fixed time, fast-forwarding, pause points, and manual ticking. That makes it unusually well suited for deterministic browser-side transition tests where the same scene needs to be asserted at specific time slices. [8]

For observability, Playwright recommends traces on CI retries instead of relying only on screenshots or videos, and its trace tooling can record screenshots and DOM snapshots for later inspection. It also supports `toHaveScreenshot()` assertions for image-based regression checks. That combination is a good fit for Phase 1 evidence gathering because it supports both human review and automated guardrails. [9]

For pure plan-only tests outside the browser, Node's built-in test runner is also a strong fit. It supports subtests, stable mock timers, and controllable mocked `Date` behavior, which is useful for transition-plan execution, interruption semantics, and composition logic that should not require a browser to validate. The Node docs are explicit that mock timers are a technique for making time-dependent tests more reliable and predictable, and that mocked time can be advanced deterministically. [10]

A practical GeoSee net therefore looks like this, as a synthesized recommendation rather than a claim about current repo coverage: browser tests for visible interpolation, legend behavior, screenshots, and traces; Node tests for plan generation, composition policy, commit/settle, and interruption semantics; and a small deck-state matrix per behavior family, carefully labeled as deck-state evidence only. The prompt's own

family list provides the starting taxonomy for that suite: section move/dissolve, MT scalar-family behavior, volume/threshold/slices, seismic families, overlays, legends/labels, camera, fog/background, commit/settle, readiness/attach, and interruption/replan. filecite turn0file0 L116-L130 [11]

Best current harden-vs-rewrite call

The best current call is **incremental hardening first, rewrite only on evidence**. That is an inference, not a final Phase 1 verdict, because the prompt itself requires live reproductions before a definitive recommendation can be made. Even so, the prompt's framing strongly favors this default: it says recent targeted fixes already landed, asks those fixes to be verified before being re-litigated, and says architecture work before residual-defect knowledge is backwards. filecite turn0file0 L26-L31 filecite turn0file0 L96-L108

The practical reason this inference makes sense is that the requirements already look like a runtime that **can** be hardened if three things become explicit and well-tested: one transition clock, explicit per-channel composition policy, and explicit lifecycle ownership for interruption/commit/settle. None of those imply a mandatory greenfield engine by themselves. They imply the need for disciplined verification and a much stronger synthetic-family test net. [12]

The threshold for switching from incremental hardening to re-architecture should therefore be evidence-based. A rewrite becomes justified only if Phase 1 shows that the same failure primitive crosses multiple behavior families, or that the current runtime cannot express expected behavior as a deterministic plan driven by a single clock with explicit composition and lifecycle ownership. Until that happens, the cheapest decision-useful path is to verify the landed fixes, publish the open defect families with hard evidence, publish the clean list, and wire those families into deterministic regression coverage. That is the exact convergence logic the prompt is asking for. filecite turn0file0 L137-L149 [13]

Deliverable created

A downloadable Markdown version of this research memo is available here:

Download the Markdown report

This session could not write directly to `C:\GeoSee\GeoSee_Dev\Transition Architecture\Phase_1_Diagnosis`, so the artifact was created as a downloadable `.md` instead. The file captures the same provisional conclusion as this summary: **use the attached Phase 1 brief exactly as written, treat the regression-test net as the main deliverable, and default to incremental hardening unless live evidence from the active repo demonstrates that the remaining failures are truly structural.** filecite turn0file0 L169-L178

---

[1] [2] [12] Window: requestAnimationFrame() method - Web APIs | MDN
https://developer.mozilla.org/en-US/docs/Web/API/Window/requestAnimationFrame

[3] [4] [13] Web Animations
https://www.w3.org/TR/web-animations-1/

5 State Chart XML (SCXML): State Machine Notation for Control Abstraction
https://www.w3.org/TR/scxml/

6 threejs.org
https://threejs.org/docs/api/en/helpers/PlaneHelper.html

7 11 Best Practices | Playwright
https://playwright.dev/docs/best-practices

8 Clock | Playwright
https://playwright.dev/docs/clock

9 Trace viewer | Playwright
https://playwright.dev/docs/trace-viewer

10 Test runner | Node.js v26.3.0 Documentation
https://nodejs.org/api/test.html